

PYTHON BASICS

You'll begin this workbook by exploring the basic building blocks of programming: writing expressions, experimenting with the interactive shell, and creating your first program. You'll also work with concepts such as values, data types, and a few functions, such as `print()` and `input()`.



LEARNING OBJECTIVES

- Experiment with instructions in the interactive shell.
- Identify the expressions, values, operators, and data types in code.
- Understand how to write strings and do string concatenation or replication.
- Use variables to store values and include variables in expressions.
- Become comfortable encountering errors and reading error messages.
- Write, run, and dissect the individual parts of a program.
- Call functions such as `print()`, `input()`, `len()`, `abs()`, and `round()`.
- Discover and convert data types with the `type()`, `int()`, `float()`, and `str()` functions.



Practice Questions

The following questions cover your knowledge of technical terms and basic concepts. For many of these questions, you can get the answer by entering the code into the interactive shell. This isn't "cheating"; rather, it's how many professional software developers verify that the code they write actually works in the way they intended. Think of the interactive shell as a way you can double-check your work.

Entering Expressions into the Interactive Shell

Entering code into the interactive shell lets you experiment with running instructions one at a time. At the `>>>` prompt, you can enter expressions made up of values, operators, and other Python code. After running the code, the interactive shell prints the result.

Match the names for questions 1 through 7 to these math operators:

`+` `-` `*` `/` `**` `//` `%`

1. Division

2. Multiplication

3. Subtraction
4. Modulo
5. Addition
6. Exponentiation
7. Floor division

An expression is the most basic kind of programming instruction in the language. Expressions consist of values (such as 2) and operators (such as +), and they can always evaluate (that is, reduce) down to a single value.

8. Is there a difference in how Python interprets these two expressions?

`2 + 2` and `2 + 2`

9. If the expression `26 / 8` evaluates to 3.25, what does the expression `26 // 8` evaluate to?

10. 26 divided by 8 is 3 with a remainder of 2. What does the expression `26 % 8` evaluate to?

11. Write the expression that adds the numbers 1 to 10. (Hint: It begins with `1 + 2 + 3 +` and so on.)

Which of the two operators in the following expressions is evaluated first according to Python's order of operation rules?

12. `(4 + 5) * 6`

13. `2 ** 3 + 1`

14. `1 + 2 ** 3`

15. `(1 + 2) ** 3`

16. `2 + 4 + 6`

Which of the following expressions produce errors? (You can enter them into the interactive shell to check.)

17. `2 +`

18. `42`

19. `((3 + 1) * 2)`

20. `((3 + 1 * 2)`

21. `(0)`

22. `1 + 2 3`

The Integer, Floating-Point, and String Data Types

A data type is a category for values, and every value belongs to exactly one data type. The integer (or int) data type includes values that are whole numbers. Numbers with a decimal point, such as 3.14, are called floating-point numbers (or floats). Text values are called strings, or strs (pronounced *stirs*).

Label the data types of the values in questions 23 through 29 as either int, float, or string. Hint: You can pass them to the `type()` function in the interactive shell to find the answers, such as `type(2)` or

```
type('hello').
```

23. 2

24. -2

25. 2.0

26. 'hello'

27. 2.2

28. '2'

29. '2.2'

30. What is the difference between the values 10, 10.0, and '10'?

String Concatenation and Replication

When the + operator combines two string values, it joins the strings as the string concatenation operator. When the * operator is used on one string value and one integer value, it becomes the string replication operator. What do the following expressions evaluate to?

31. 'Hello' + 'Hello' + 'Hello'

32. 'Hello' * 3

33. 3 * 'Hello'

34. (2 * 2) * 'Hello'

35. '13' + '12'

Which of the following expressions produce errors?

36. 'Forgot the closing quote

37. 'Hello' * 3.0

38. 'Hello' + 3

39. Hello + Hello + Hello

40. 'Alice' * 'Bob'

41. 'Hello' / 5

42. 'Hello' / 'Hello'

Storing Values in Variables

A variable is like a box in the computer's memory that can store a single value. If you want to use the result of an evaluated expression later in your program, you can save it inside a variable. There are a few rules variable names must follow, and a good variable name describes the data the variable contains.

The following programs store values in variables. Determine what each program outputs.

43.

```
nephew = 'Jack'
print(nephew)
```

44.

```
nephew = 'Jack'
print('nephew')
```

45.

```
nephew = 'Jack'
nephew = 'Albert'
print(nephew)
```

46.

```
nephew = 'Jack'
Nephew = 'Albert'
print(nephew)
```

Why do the following programs cause an error?

47.

```
nephew = Jack
print(nephew)
```

48.

```
nephew = 'Jack'
print(Jack)
```

49.

```
nephew = 'Jack'
print(NEPHEW)
```

50. `print(nephew)`

Which of the following are valid variable names?

51. `number_of_cats`

52. `number-of-cats`

53. `numberofcats`

54. `numberOfCats`

55. `_42`

56. `_`

57. `42`

Your First Program

While the interactive shell is good for running Python instructions one at a time, to write entire Python programs you'll need to enter the instructions into the file editor. Chapter 1 of *Automate the Boring Stuff with Python* includes a “Hello, world” program that uses comments, the `print()` and `input()` functions, and

the value and operator concepts from the previous section. To further explore these building blocks of programs, label the following as a variable, function call, or string.

```
58. 'hello'  
59. hello  
60. print()  
61. 'print()'
```

Do the following expressions cause an error or no error? If they cause no error, what do they evaluate to?

```
62. int('42')  
63. int('forty two')  
64. int('Hello')  
65. int(-42)  
66. int(3.1415)  
67. float(-42)  
68. str(-42)  
69. str(3.1415)  
70. str('Hello')  
71. str(float(int(3.14)))  
72. str(3)  
73. str(3.0)
```

Answer the following questions to further your understanding of the techniques used in the “Hello, world” program.

74. Why does this two-line program cause an error?

```
number_of_cats = 4  
print('I have ' + number_of_cats)
```

75. Does `round(4.9)` evaluate to the integer 5 or the float 5.0?

76. Describe what the `abs()` function returns.

77. What does `abs(5)` return?

78. What does `abs(-5)` return?

How Computers Store Data with Binary Numbers

Binary, also called the *base-2 number system*, can represent all of the same numbers that our more familiar base-10 decimal number system can. Decimal has 10 digits, 0 through 9, but binary has only 0 and 1. By

representing text, image, audio, and other kinds of data as binary numbers, computers can store and process the data. Answer the following questions about data representation.

- 79. Why do computers use the base-2 binary number system instead of the more familiar base-10 decimal system humans use?
- 80. How many bits are in 1 byte?

Determine how many bytes the units in 81 through 84 represent, both as an exponent like 2^{10} and a whole number like 1,024. You can enter an expression like `2 ** 10` into the interactive shell to calculate 2^{10} .

- 81. Kilobyte
- 82. Megabyte
- 83. Gigabyte
- 84. Terabyte
- 85. Decimal 2 is 10 in binary. What is decimal 3 in binary?
- 86. Decimal 7 is 111 in binary. What is decimal 8 in binary?



Practice Projects

The following practice projects will illustrate the concepts you've learned so far.

Rectangle Printer

Write a program that prints a rectangle of capital *O* characters. For example, the following rectangle has a width of eight and a height of five:

```
00000000
00000000
00000000
00000000
00000000
```

The program should always print a rectangle of *O* characters that has a height of five (that is, five rows), but the width should be based on an integer the user enters. For example, the output of this program could look like this:

```
Enter the width for the rectangle:
15
000000000000000
000000000000000
000000000000000
000000000000000
000000000000000
```

Here are some hints to help you write this program:

- Call the `print()` function with a message to tell the user to enter the width, then call `input()` to accept the width.
- Store the width returned from `input()` in a variable.
- The `input()` function returns strings, but we want an integer form of the user input, so pass the variable to the `int()` function and store what `int()` returns in a variable.
- Use string replication to create a string of *O* letters of the desired width. (If a variable named `width` has 8 in it, then `0 * width` will evaluate to 00000000.)
- Call `print()` five times to produce five rows of the string replication letters.

Save this program in a file named *rectPrint.py*.

Perimeter and Area Calculator

Write a program that accepts the width and length of a rectangular space from the user and then calculates both the perimeter and area of this space. For example, the output of the program could look like this:

Enter the width for the rectangle:

9

Enter the length for the rectangle:

5

Area of the rectangle:

45

Perimeter of the rectangle:

28

Here are some hints to write this program:

- Call the `print()` function with a message that tells the user to enter the width, and then call `input()` to accept the width. Do the same for the length.
- Store the width and length returned from `input()` in two separate variables.
- The `input()` function returns strings, but we want an integer form of the user input, so pass the variable to the `int()` function and store what `int()` returns in a variable.
- The perimeter is the sum of twice the width and twice the length. The area is the width multiplied by the length.

Save this program in a file named *perimeterAreaCalculator.py*.